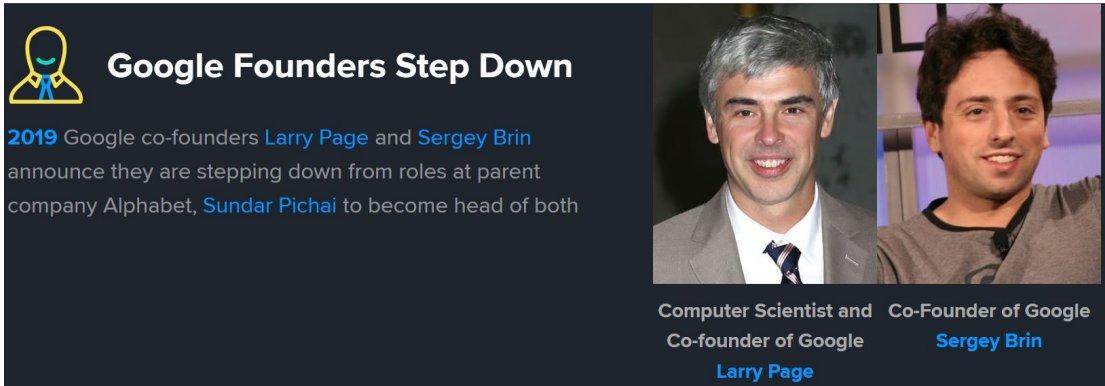


COMS W4156 Advanced Software Engineering (ASE)

December 3, 2024

On this day ...



Agenda



1. Shreya Kochar
2. Amy Pu
3. Sreehari Rammohan

as time permits:

4. continue test oracles
5. start test suite quality
6. finishing the semester

Shreya Kochar

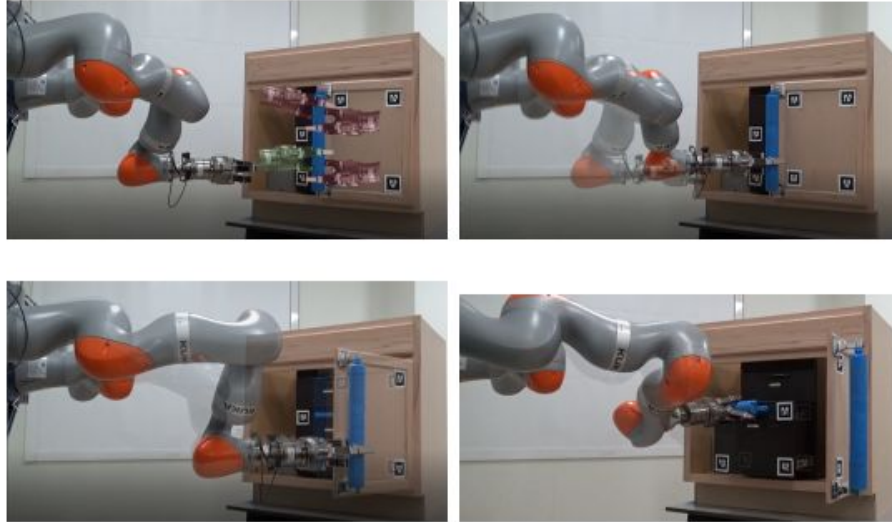
[slides](#)



Amy Pu



Sreehari Rammohan



Agenda

1. Shreya Kochar
 2. Amy Pu
 3. Sreehari Rammohan
- as time permits:
4. **continue test oracles**
 5. start test suite quality
 6. finishing the semester

Defining A Test Oracle

The mechanism or principle by which we determine whether or not the software under test is behaving reasonably.

- *Unreasonable SUT behavior requires investigation by a person.*
- *When the oracle says the SUT behavior is unremarkable, we just move on.*



Reprise: What is a Test Oracle?

An entity that already knows the expected outputs for given inputs and/or can check that the actual outputs are the same as or consistent with the expected outputs



Often the human tester is the test oracle - either they know the correct output in advance or, given an actual output, can easily figure out whether or not it is correct

For manual testing, the human tester checks the results. For automated testing, the human tester writes assertions to automatically check the results

Sometimes There Isn't Any Test Oracle

A frequently invoked assumption in program testing is that there is an oracle (i.e. the tester or an external mechanism can accurately decide whether or not the output produced by a program is correct). A program is non-testable if either an oracle does not exist or the tester must expend some extraordinary amount of time to determine whether or not the output is correct. The reasonableness of the oracle assumption is examined and the conclusion is reached that in many cases this is not a realistic assumption. The consequences of assuming the availability of an oracle are examined and alternatives investigated. ([Weyuker 1982](#))



Pseudo-oracle = one test case acts as an oracle for another test case. You may not know whether the result of either test case is right, but if the results are inconsistent you know (at least) one of them is wrong. They could both be wrong

Pseudo-Oracles: Regression Testing

Regression testing is the most common form of pseudo-oracle. The test results for the old version are known and we re-run the tests on the new version to see if they have the same or different results

Some test cases *should* change results on the new version $\Rightarrow$ when we try to fix a bug that caused a test case to fail on the old version, we want that test case to pass on the new version

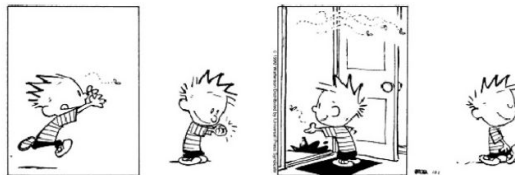
But unrelated test cases should not be affected $\Rightarrow$ if they are, that indicates a hidden dependency and (probably) a newly introduced bug

```
program_v1(input) -> output_v1
```

```
program_v2(input) -> output_v2
```

```
output_v1 =? output_v2
```

Regression:
“when you fix one bug, you introduce several newer bugs.”



Pseudo-Oracles: Differential Testing

Differential testing requires a second *independently developed* implementation of the same functionality, as well as a test inputs applicable to both

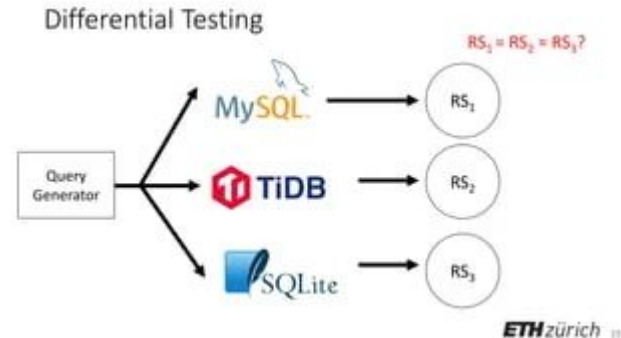
Practical for standard protocols, e.g., there are many implementations of HTTP

- If they disagree, one or both is wrong
- If they agree, that does not guarantee both are right, but does provide some confidence that they are both right

`program_1(input) -> output_1`

`program_2(input) -> output_2`

`output_1 =? output_2`



Pseudo-Oracles: Metamorphic Testing

Metamorphic testing requires only one implementation. Starts with some original input and its known original output. We do not need to know whether this output is correct

Create a new test case by *deriving* a new input from the original input, and *predicting* the expected new output from the original input, the derived input and the original output - the prediction is usually that the output is the same, nearly the same, or changed in a simple way

If the actual new output deviates too much from the predicted output, there is (probably) a bug

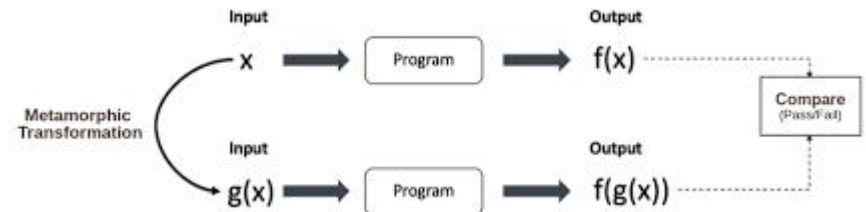
```
program(input_1) -> output_1
```

```
deriveInput(input_1) -> input_2
```

```
predictOutput(input_1, input_2,  
output_1) -> predicted_output_2
```

```
program(input_2) -> output_2
```

```
check(predicted_output_2, output_2)  
-> do they match?
```



Predicted Output is not Necessarily the Same Output

Metamorphic relations for a sorting program (lowest first)
applied to an array of numbers
`sort(5,2,3,1,4) -> 1,2,3,4,5`

Permutative: if we shuffle the order of the input array, the
sorted output array should be the same
`sort(3,4,1,2,5) -> 1,2,3,4,5` 😊

Additive: if we add N to every element of the input array,
the sorted output array should be in the same order
`sort(25,22,23,21,24) -> 21,22,23,24,25` 😊

Multiplicative: if we multiply every element of the input
array by positive N, the sorted output array should be in
the same order
`sort(15,6,9,3,12) -> 3,6,9,12,15` 😊

Invertive: if we multiply every element of the input array
by negative N, the sorted output array should be in the
opposite order
`sort(-5,-2,-3,-1,-4) -> -5,-4,-3,-2,-1` 😊

Inclusive: if we add one new element to the input array,
the sorted output array should be the same except for
the placement of that new element
`sort(5,2,42,3,1,4) -> 1,2,3,4,5,42` 😊

Exclusive: if we remove one element from the input
array, the sorted output array should be the same except
it is missing that one element
`sort(5,3,1,4) -> 1,3,4,5` 😊

Using Metamorphic Relations to Detect Bugs

Original

`buggysort(5,2,3,1,4) -> 1,2,3,4,5`

Permutative: if we shuffle the order of the input array, the sorted output array should be the same

`buggysort(3,4,1,2,5) -> 2,4,1,5,3` 😞

Additive: if we add N to every element of the input array, the sorted output array should be in the same order

`buggysort(25,22,23,21,24) -> 25,25,25,25,25` 😬

Multiplicative: if we multiply every element of the input array by (positive) N, the sorted output array should be in the same order

`buggysort(15,6,9,3,12) -> 6,12,3,15,9` 😞

Invertive: if we multiply every element of the input array by (negative) N, the sorted output array should be in the opposite order

`buggysort(-5,-2,-3,-1,-4) -> -1,-4,-3,-2,-5` 😞

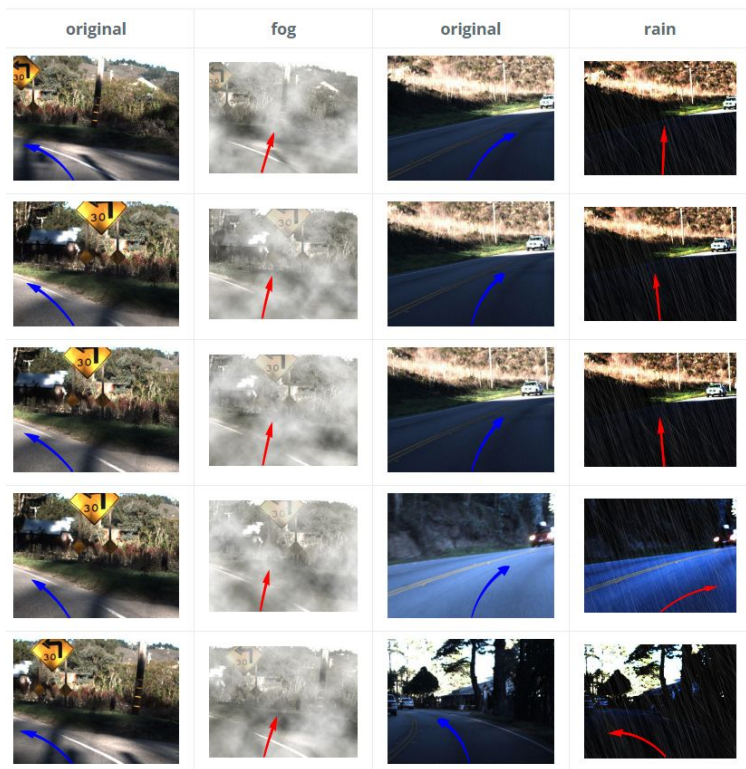
Inclusive: if we add one new element to the input array, the sorted output array should be the same except for the placement of that new element

`buggysort(5,2,42,3,1,4) ->` ⌚ 😡

Exclusive: if we remove one element from the input array, the sorted output array should be the same except it is missing that one element

`buggysort(5,3,1,4) ->` 💣 😬

Metamorphic Testing Examples



Say we have a machine learning application that controls the steering of a [self-driving car](#), so it obeys traffic laws and doesn't cause accidents

How do we know what is “correct” steering for a given real-world driving scenario? How can we write test cases?

Different human drivers will not necessarily steer at exactly the same angle and the same human driver will not necessarily steer at the same angle every time

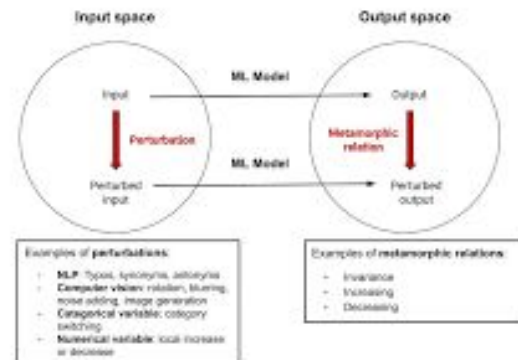
[DeepTest](#): Synthetic but realistic changes to driving images such as adding snow, fog, rain, more sunlight, less sunlight, etc. should not (in most cases) change automobile steering

Where Do Metamorphic Relations Come From?

Some are generic, e.g., permutation can be applied to any ordered input

Others are specific to domain or application such as car steering

Metamorphic Testing automates generation of additional tests from existing tests whether or not there is a test oracle - it's a test generation technique as well as a pseudo-oracle technique



Agenda

1. Shreya Kochar
2. Amy Pu
3. Sreehari Rammohan

as time permits:

4. continue test oracles
5. **start test suite quality**
6. finishing the semester



Test Suite Quality

An empty test suite has 0% coverage but all its tests pass!

A non-empty test suite consisting entirely of valid inputs, where all the tests pass, may have 100% coverage but low quality because the code is not checking for invalid inputs

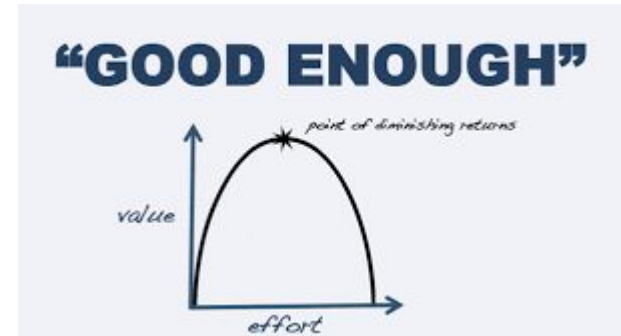
Remember no test suite can cover code that has not been written



Testing the Test Suite

How do we know whether we have a “good” or “good enough” test suite?

- ✓ Equivalence Partitioning (and Boundary Analysis) - *for outputs as well as inputs*
- ✓ Control Flow Coverage - high percentage of statement, branch, [MC/DC](#) & [MCC](#)
- ✓ [Data Flow Coverage](#) - high percentage of intraprocedural paths from each variable or field definition (write) to everywhere the written value is used (read) before it is overwritten
- ✓ [Mutation Testing](#) - use mutation operators to inject bugs into the code (e.g., switch > to <, insert return statements at arbitrary locations in the middle of functions) and run the regular test suite, did it “kill” (detect) all the mutants?



Equivalence Partitioning for *Inputs*

Inputs include any parameters that are read, any global in-memory state that is read, any environment data that is read (from files, databases, network, devices, environment variables, etc.), *and the return values and other shared state received from API, library and system calls that are read* (used for anything) by the code under test.

Equivalence partitioning of parameters is not enough, also need to test input equivalence partitions for global variables, environment data, and outputs from any external calls that are, effectively, inputs to the code under test. Forcing some of these equivalence partitions may require mocks.

```
myClass foo (myClass obj, something x)
{
    local anotherThing y;

    some code;
    y = systemCall(x);
    some more code;
    obj.field = readFromFile(y);
    code that does something;

    return obj;
}
```

Equivalence Partitioning for *Outputs*

```
myClass foo (myClass obj, something x)
{
    local anotherThing y;

    some code;
    y = systemCall(x);
    some more code;
    obj.field = readFromFile(y);
    code that does something;

    return obj;
}
```

Outputs include any (object, by-reference or data structure) parameters that are written, any global in-memory state that is written, any environment data that is written (files, databases, network, devices, environment variables, etc.), *and the parameters and shared state sent to API, library and system calls* by the code under test.

Equivalence partitioning of return values is not enough, also need to test output equivalence partitions for updated parameters, global variables, environment data, and inputs to any external calls. Forcing output equivalence partitions typically requires specially crafted inputs that lead to generating those outputs.

Control Flow Coverage

Multiple Condition Coverage (MCC)
and Modified Condition/Decision
Coverage (MC/DC) rarely used
outside safety-critical software

MC/DC: each individual condition is
true at least once and false at least
once [linear]

MCC: conditions must evaluate to T
or F in all combinations
[combinatoric]

```
if ( (A and B) or (C and D) )  
    { do something }  
else  
    { do something else }
```

```
if ( (A or B) and (C or D) )  
    { do something }  
else  
    { do something else }
```

Example

```
boolean purchaseAlcohol  
(int buyerAge, int ageFriend)  
{  
    boolean allow = False;  
    if ((buyerAge>=21) or  
        (ageFriend>=21))  
    { allow = True; }  
    return allow;  
}
```

Assert purchaseAlcohol(25,25) == True
gives 100% *statement* coverage

Assert purchaseAlcohol(25,25) == True
Assert purchaseAlcohol(20,20) == False
gives 100% *branch* coverage

But *both* the buyer *and* the friend must be
21 or older

Does MCDC coverage detect the bug?

Example continued

```
boolean purchaseAlcohol  
(int buyerAge, int ageFriend)  
{  
    boolean allow = False;  
    if ((buyerAge>=21) or  
        (ageFriend>=21))  
    { allow = True; }  
    return allow;  
}
```

purchaseAlcohol(25,25) == True covers T, T
purchaseAlcohol(20,20) == False covers F, F
purchaseAlcohol(25,20) == False covers T, F
purchaseAlcohol(20,25) == False covers F, T

Does a set of test cases fulfilling MCC
coverage (truth table) detect the bug?

Data Flow Coverage

`x := 12345 ;`

lots of code that does not change x

`y := derive some value from x ;`

some code that changes x

`z := derive some value from x;`

Cover all paths from a variable definition (write) to any uses (reads) of that variable *before* its value is overwritten = a new definition

Data flow is about reads and writes, not variable declarations

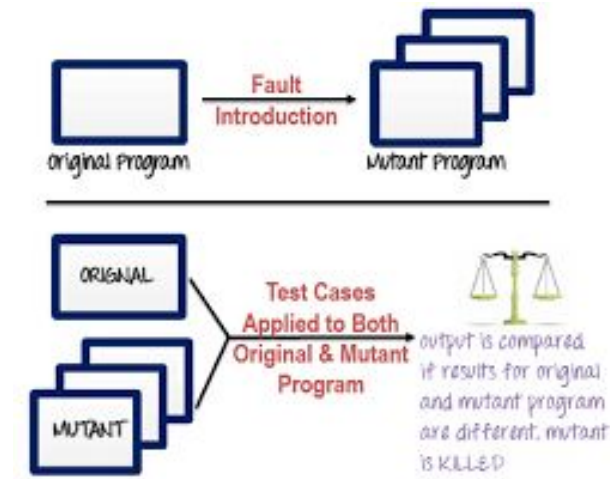
→ More general "coverage": Create some graph representation of your program, write tests that exercise every edge

Mutation Testing

Intuition: Does your test suite actually detect bugs? Check by changing your code to intentionally insert a bug, then run your test suite. Did any test that passed with the original code and exercises the changed code now fail with the modified code? If not, then your test suite is not good enough to detect the bug you inserted. Repeat with another bug, etc.

Example mutation testing tools:

[PIT](#) for Java, [Mull](#) for C and C++



Mutation Operators

Statement deletion

Statement duplication or insertion, e.g., `return 0;`

Replacement of boolean subexpressions with *true*, *false*

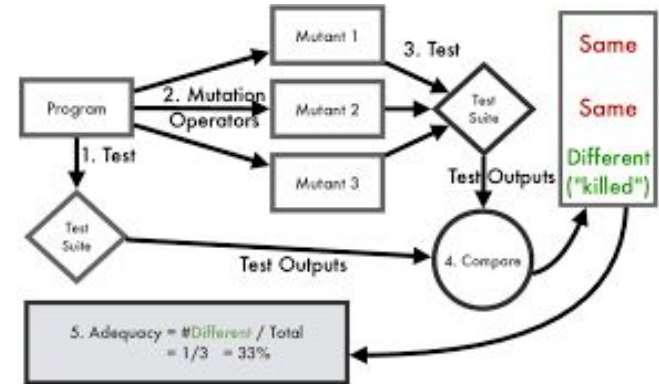
Replacement of some arithmetic operations with others,
e.g., `+` with `*`, `-` with `/`

Replacement of some boolean relations with others,
e.g., `>` with `>=`, `==` and `<=`

Replacement of variables with others from the same scope
(variable types must be compatible)

Changes in synchronization concerned with concurrency and
multi-threading

Insert Security Vulnerability (from [CVEs](#) and [CWEs](#))



Mutation Testing Relies On...

- “Competent Programmer Hypothesis”: Most software faults introduced by experienced programmers are due to small errors
- “RIP Model”:
 1. A test must *reach* the mutated statement
 2. Test input data should *infect* the program state by causing different program states for the mutant vs. the original program
 3. The incorrect program state must *propagate* to the program's (or method's) output so it can be checked by the test



Agenda

1. Shreya Kochar
2. Amy Pu
3. Sreehari Rammohan

as time permits:

4. continue test oracles
5. start test suite quality

6. finishing the semester



Finishing the Semester

[Second Iteration Demo](#) due December 6 (this Friday)

Second Assessment Friday December 6 12:01AM to Monday December 9 11:59PM (four days starting this Friday)

➤ focus on material October 22 and later

[Demo Day](#) Monday December 16 10AM to 4PM (Monday of finals week)



Thursday

More research talks

Remaining testing topics



Ask Me Anything

